Table Partitioning in PostgreSQL

Table of Contents

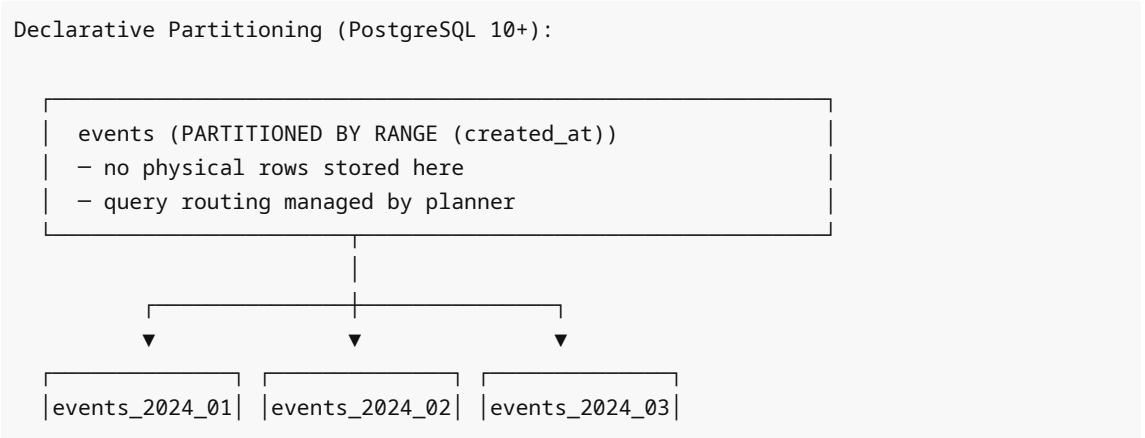
Learning Objectives

By the end of this module you will be able to:

- Choose the correct partitioning strategy for a given workload
- Create range, list, and hash partitioned tables with proper structure
- Understand and verify partition pruning with EXPLAIN
- Implement partition maintenance including adding, detaching, and dropping partitions
- Configure pg_partman for automatic partition management
- Design sub-partitioning strategies for large time-series datasets

---

Partitioning Architecture Overview



```
  │ Jan 2024     │ │ Feb 2024     │ │ Mar 2024     │
  │ (real data)  │ │ (real data)  │ │ (real data)  │
  └──────────────┘ └──────────────┘ └──────────────┘

  Query: SELECT * FROM events WHERE created_at >= '2024-02-01'
                                AND created_at <  '2024-03-01'
  Result: Only scans events_2024_02 (partition pruning)
```

When to partition

- Tables exceeding 100GB where time-based deletion is needed
- Tables where queries always filter on a specific column (partition key)
- When you need to drop data in bulk (DROP vs DELETE — orders of magnitude faster)
- When you need data tiering (hot data on SSD, cold on spinning disk)

When NOT to partition

- Small tables (< 10GB) — overhead outweighs benefits
- When query patterns are unpredictable and don't filter on the partition key
- When foreign key references need to point to individual partitions
- When you need cross-partition `UNIQUE` constraints on non-partition-key columns

---

Range Partitioning

Range partitioning assigns rows to partitions based on the partition key being within a range.

```sql
-- Time-series events table
CREATE TABLE events (
    id          BIGSERIAL,
    user_id     INT NOT NULL,
    event_type  TEXT NOT NULL,
    payload     JSONB DEFAULT '{}',
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
) PARTITION BY RANGE (created_at);

-- Create monthly partitions
CREATE TABLE events_2024_01 PARTITION OF events
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

CREATE TABLE events_2024_02 PARTITION OF events
    FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');

CREATE TABLE events_2024_03 PARTITION OF events
    FOR VALUES FROM ('2024-03-01') TO ('2024-04-01');

CREATE TABLE events_2024_04 PARTITION OF events
    FOR VALUES FROM ('2024-04-01') TO ('2024-05-01');

-- Add default partition to catch out-of-range values
CREATE TABLE events_default PARTITION OF events DEFAULT;

-- Test: Insert data
```

```sql
INSERT INTO events (user_id, event_type, created_at) VALUES
(1, 'login',    '2024-01-15 10:00:00+00'),
(2, 'purchase', '2024-02-22 14:30:00+00'),
(3, 'logout',   '2024-03-05 09:00:00+00');

-- Verify data went to correct partitions
SELECT tableoid::regclass AS partition, count(*)
FROM events
GROUP BY tableoid;
```

Range partitioning on numeric key

```sql
CREATE TABLE orders (
    order_id   BIGINT NOT NULL,
    customer_id INT NOT NULL,
    total       NUMERIC(12,2),
    status      TEXT
) PARTITION BY RANGE (order_id);

CREATE TABLE orders_p1 PARTITION OF orders FOR VALUES FROM (1)        TO (1000001);
CREATE TABLE orders_p2 PARTITION OF orders FOR VALUES FROM (1000001)  TO (2000001);
CREATE TABLE orders_p3 PARTITION OF orders FOR VALUES FROM (2000001)  TO (3000001);
```

Multi-column range partitioning

```sql
CREATE TABLE sensor_data (
    sensor_id   INT NOT NULL,
    recorded_at DATE NOT NULL,
    value       FLOAT
) PARTITION BY RANGE (recorded_at, sensor_id);

CREATE TABLE sensor_data_2024_q1_low PARTITION OF sensor_data
    FOR VALUES FROM ('2024-01-01', 1) TO ('2024-04-01', 100);
```

List Partitioning

List partitioning assigns rows to partitions based on matching a specific list of values.

```sql
-- Partition by region
CREATE TABLE customers (
    id         BIGSERIAL,
    name       TEXT NOT NULL,
    email      TEXT NOT NULL,
    region     TEXT NOT NULL,
    plan       TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
) PARTITION BY LIST (region);
```

```
CREATE TABLE customers_us PARTITION OF customers
    FOR VALUES IN ('US', 'CA', 'MX');

CREATE TABLE customers_eu PARTITION OF customers
    FOR VALUES IN ('GB', 'DE', 'FR', 'IT', 'ES', 'NL', 'SE');

CREATE TABLE customers_apac PARTITION OF customers
    FOR VALUES IN ('JP', 'AU', 'SG', 'IN', 'KR');

CREATE TABLE customers_other PARTITION OF customers DEFAULT;

-- Insert test data
INSERT INTO customers (name, email, region) VALUES
('Alice Smith', 'alice@example.com', 'US'),
('Hans Müller', 'hans@example.de',   'DE'),
('Yuki Tanaka', 'yuki@example.jp',   'JP');

-- Query a specific partition directly (bypasses routing)
SELECT * FROM customers_eu WHERE email LIKE '%@example.de';

-- Or query parent (routes automatically)
SELECT tableoid::regclass, name, region FROM customers;
```

Hash Partitioning

Hash partitioning distributes rows based on a hash of the partition key. Use when you have no natural range or list boundary but want to distribute load evenly.

```
-- Distribute users across 8 partitions
CREATE TABLE user_sessions (
    session_id  UUID NOT NULL DEFAULT gen_random_uuid(),
    user_id     INT NOT NULL,
    data        JSONB DEFAULT '{}',
    expires_at  TIMESTAMPTZ
) PARTITION BY HASH (user_id);

CREATE TABLE user_sessions_p0 PARTITION OF user_sessions
    FOR VALUES WITH (MODULUS 8, REMAINDER 0);
CREATE TABLE user_sessions_p1 PARTITION OF user_sessions
    FOR VALUES WITH (MODULUS 8, REMAINDER 1);
CREATE TABLE user_sessions_p2 PARTITION OF user_sessions
    FOR VALUES WITH (MODULUS 8, REMAINDER 2);
CREATE TABLE user_sessions_p3 PARTITION OF user_sessions
    FOR VALUES WITH (MODULUS 8, REMAINDER 3);
CREATE TABLE user_sessions_p4 PARTITION OF user_sessions
    FOR VALUES WITH (MODULUS 8, REMAINDER 4);
CREATE TABLE user_sessions_p5 PARTITION OF user_sessions
    FOR VALUES WITH (MODULUS 8, REMAINDER 5);
CREATE TABLE user_sessions_p6 PARTITION OF user_sessions
    FOR VALUES WITH (MODULUS 8, REMAINDER 6);
```

```
CREATE TABLE user_sessions_p7 PARTITION OF user_sessions
    FOR VALUES WITH (MODULUS 8, REMAINDER 7);

-- Verify distribution
SELECT tableoid::regclass AS partition, COUNT(*)
FROM user_sessions
GROUP BY tableoid
ORDER BY partition;
```

Partition Pruning

Partition pruning is the planner optimization that skips scanning irrelevant partitions.

```
-- Populate events table with test data
INSERT INTO events (user_id, event_type, created_at)
SELECT
    (random() * 10000)::int,
    CASE (random()*3)::int WHEN 0 THEN 'login' WHEN 1 THEN 'purchase' ELSE 'logout'
END,
    '2024-01-01'::timestamptz + (random() * 90)::int * interval '1 day'
FROM generate_series(1, 100000);

-- Pruning in action: query only Feb events
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT COUNT(*) FROM events
WHERE created_at >= '2024-02-01'
  AND created_at <  '2024-03-01';

/*
Expected output:
  Aggregate
    -> Append
         -> Seq Scan on events_2024_02  <-- only this partition scanned!
              Filter: (created_at >= '2024-02-01' AND created_at < '2024-03-01')
  Partitions selected: 1 out of 4
*/

-- Pruning with parameterized query (runtime pruning)
PREPARE get_events(timestamptz, timestamptz) AS
    SELECT * FROM events WHERE created_at BETWEEN $1 AND $2;

EXPLAIN (ANALYZE)
EXECUTE get_events('2024-03-01', '2024-03-31');
-- enable_partition_pruning must be on (default)

SHOW enable_partition_pruning;  -- should be 'on'

-- Pruning fails if key is transformed
-- BAD: no pruning (function wraps the key)
EXPLAIN SELECT * FROM events
```

```
WHERE DATE_TRUNC('month', created_at) = '2024-02-01';

-- GOOD: pruning works (key compared directly)
EXPLAIN SELECT * FROM events
WHERE created_at >= '2024-02-01' AND created_at < '2024-03-01';
```

Partition-Wise Joins and Aggregations

```
-- Enable partition-wise operations
SET enable_partitionwise_join = on;
SET enable_partitionwise_aggregate = on;

-- Two partitioned tables joined
CREATE TABLE event_metadata (
    event_id    BIGINT NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL,
    meta_key    TEXT,
    meta_value  TEXT
) PARTITION BY RANGE (created_at);

CREATE TABLE event_metadata_2024_01 PARTITION OF event_metadata
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
CREATE TABLE event_metadata_2024_02 PARTITION OF event_metadata
    FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');
CREATE TABLE event_metadata_2024_03 PARTITION OF event_metadata
    FOR VALUES FROM ('2024-03-01') TO ('2024-04-01');

-- Partition-wise join: joins matching partitions directly
EXPLAIN (ANALYZE, FORMAT TEXT)
SELECT e.event_type, em.meta_key, em.meta_value
FROM events e
JOIN event_metadata em ON e.id = em.event_id
                      AND e.created_at = em.created_at
WHERE e.created_at >= '2024-02-01'
  AND e.created_at <  '2024-03-01';
-- With partition-wise join on: joins events_2024_02 to event_metadata_2024_02
directly

-- Partition-wise aggregate
EXPLAIN SELECT
    DATE_TRUNC('day', created_at) AS day,
    COUNT(*) AS event_count
FROM events
GROUP BY DATE_TRUNC('day', created_at)
ORDER BY day;
-- Each partition aggregates independently, then results merged
```

Indexes on Partitioned Tables

```sql
-- Global index on parent (creates index on all partitions)
CREATE INDEX idx_events_user_id ON events (user_id);
-- This creates idx_events_2024_01_user_id, idx_events_2024_02_user_id, etc.

-- Index on specific partition only
CREATE INDEX idx_events_jan_type ON events_2024_01 (event_type);

-- Unique constraint (must include partition key)
CREATE UNIQUE INDEX ON events (user_id, created_at, id);
-- Cannot have unique index on non-partition-key columns across partitions

-- Primary key must include partition key
ALTER TABLE events ADD PRIMARY KEY (id, created_at);
-- For events with separate BIGSERIAL id, this is often a gotcha

-- Verify index propagation
SELECT
    indexname,
    tablename,
    pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes
WHERE tablename LIKE 'events%'
ORDER BY tablename;
```

Partition Maintenance Scripts

Adding a new partition

```sql
-- Function to create next month's partition
CREATE OR REPLACE FUNCTION create_monthly_partition(
    p_table TEXT,
    p_year  INT,
    p_month INT
) RETURNS void LANGUAGE plpgsql AS $$
DECLARE
    v_partition_name TEXT;
    v_start          DATE;
    v_end            DATE;
BEGIN
    v_start := make_date(p_year, p_month, 1);
    v_end   := v_start + INTERVAL '1 month';
    v_partition_name := p_table || '_' || TO_CHAR(v_start, 'YYYY_MM');

    EXECUTE format(
        'CREATE TABLE IF NOT EXISTS %I PARTITION OF %I
         FOR VALUES FROM (%L) TO (%L)',
        v_partition_name, p_table, v_start, v_end
    );
```

```
    -- Create indexes on the new partition
    EXECUTE format(
        'CREATE INDEX IF NOT EXISTS %I ON %I (user_id)',
        v_partition_name || '_user_id_idx', v_partition_name
    );

    RAISE NOTICE 'Created partition: %', v_partition_name;
END;
$$;

-- Create next 3 months of partitions
SELECT create_monthly_partition('events', 2024, m)
FROM generate_series(5, 7) AS m;
```

Detaching and archiving old partitions

```
-- Step 1: Detach partition (fast, metadata operation)
ALTER TABLE events DETACH PARTITION events_2024_01;
-- Data is now in events_2024_01 as a standalone table

-- Step 2: Archive to cold storage (optional: move to tablespace)
ALTER TABLE events_2024_01 SET TABLESPACE cold_storage;

-- Step 3: Drop detached partition
DROP TABLE events_2024_01;

-- Detach in parallel/CONCURRENTLY (PostgreSQL 14+)
ALTER TABLE events DETACH PARTITION events_2024_01 CONCURRENTLY;
-- Non-blocking: allows reads/writes during detach

-- Attach existing table as new partition
CREATE TABLE events_archive (LIKE events INCLUDING ALL);
-- ... populate events_archive ...
ALTER TABLE events ATTACH PARTITION events_archive
    FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');
```

Partition cleanup job

```
-- Drop partitions older than N months
CREATE OR REPLACE FUNCTION drop_old_partitions(
    p_table         TEXT,
    p_retention_months INT DEFAULT 12
) RETURNS INT LANGUAGE plpgsql AS $$
DECLARE
    r             RECORD;
    v_cutoff_date DATE := DATE_TRUNC('month', now()) - (p_retention_months || '
months')::interval;
    v_dropped     INT  := 0;
BEGIN
```

```
    FOR r IN
        SELECT c.relname AS partition_name
        FROM pg_class c
        JOIN pg_inherits i ON c.oid = i.inhrelid
        JOIN pg_class p ON p.oid = i.inhparent
        WHERE p.relname = p_table
          AND c.relname < p_table || '_' || TO_CHAR(v_cutoff_date, 'YYYY_MM')
    LOOP
        RAISE NOTICE 'Dropping partition: %', r.partition_name;
        EXECUTE format('DROP TABLE IF EXISTS %I', r.partition_name);
        v_dropped := v_dropped + 1;
    END LOOP;

    RETURN v_dropped;
END;
$$;

-- Example: keep 12 months of events
SELECT drop_old_partitions('events', 12);
```

---

pg_partman Extension

`pg_partman` automates partition creation and maintenance.

```
-- Install (requires superuser or pg_partman installed by DBA)
CREATE EXTENSION IF NOT EXISTS pg_partman;

-- Create a partitioned table managed by pg_partman
SELECT partman.create_parent(
    p_parent_table  => 'public.events',
    p_control       => 'created_at',
    p_interval      => 'monthly',
    p_type          => 'range',
    p_premake       => 4  -- pre-create 4 future partitions
);

-- Check partition management config
SELECT * FROM partman.part_config WHERE parent_table = 'public.events';

-- Run maintenance (creates new partitions, cleans old ones)
CALL partman.run_maintenance_proc();

-- Automate with pg_cron (if available)
SELECT cron.schedule('partman-maintenance', '0 * * * *',
    $$CALL partman.run_maintenance_proc()$$);

-- Configure retention (automatically drop old partitions)
UPDATE partman.part_config
SET retention          = '12 months',
    retention_keep_table = false,   -- actually drop (not just detach)
```

```
    infinite_time_partitions = true
WHERE parent_table = 'public.events';

-- Manual undo partitioning (convert back to regular table)
SELECT partman.undo_partition(
    p_parent_table => 'public.events',
    p_target_table => 'public.events_consolidated',
    p_batch_count  => 10
);
```

Sub-partitioning

```
-- Events partitioned by month, then by region (sub-partitioning)
CREATE TABLE events_2024_02 PARTITION OF events
    FOR VALUES FROM ('2024-02-01') TO ('2024-03-01')
    PARTITION BY LIST (region);

CREATE TABLE events_2024_02_us PARTITION OF events_2024_02
    FOR VALUES IN ('US', 'CA');

CREATE TABLE events_2024_02_eu PARTITION OF events_2024_02
    FOR VALUES IN ('GB', 'DE', 'FR');

CREATE TABLE events_2024_02_other PARTITION OF events_2024_02 DEFAULT;
```

Default Partitions

```
-- Default partition catches all rows that don't match any explicit partition
CREATE TABLE events_default PARTITION OF events DEFAULT;

-- Check what's in the default partition (indicates missing partitions)
SELECT COUNT(*), MIN(created_at), MAX(created_at)
FROM events_default;

-- Move default data to proper partition
-- Step 1: Create new partition
CREATE TABLE events_2024_05 PARTITION OF events
    FOR VALUES FROM ('2024-05-01') TO ('2024-06-01');
-- FAILS if default partition has rows in that range!

-- Workaround: Delete from default first, or use INSERT...SELECT
BEGIN;
    -- Lock to prevent new rows during migration
    LOCK TABLE events_default IN SHARE ROW EXCLUSIVE MODE;

    -- Move rows to proper partition
    WITH moved AS (
        DELETE FROM events_default
```

```
        WHERE created_at >= '2024-05-01' AND created_at < '2024-06-01'
        RETURNING *
    )
    INSERT INTO events SELECT * FROM moved;

    -- Now safe to create partition
    CREATE TABLE events_2024_05 PARTITION OF events
        FOR VALUES FROM ('2024-05-01') TO ('2024-06-01');
COMMIT;
```

Production Monitoring Queries

```
-- List all partitions with row counts and sizes
SELECT
    c.relname                                  AS partition_name,
    pg_size_pretty(pg_relation_size(c.oid))    AS data_size,
    pg_size_pretty(pg_indexes_size(c.oid))     AS index_size,
    pg_size_pretty(pg_total_relation_size(c.oid)) AS total_size,
    s.n_live_tup                               AS live_rows,
    s.n_dead_tup                               AS dead_rows,
    s.last_vacuum,
    s.last_autovacuum,
    s.last_analyze
FROM pg_class c
JOIN pg_inherits i ON c.oid = i.inhrelid
JOIN pg_class p ON p.oid = i.inhparent
LEFT JOIN pg_stat_user_tables s ON s.relid = c.oid
WHERE p.relname = 'events'
ORDER BY c.relname;

-- Check partition boundaries
SELECT
    c.relname AS partition,
    pg_get_expr(c.relpartbound, c.oid) AS bounds
FROM pg_class c
JOIN pg_inherits i ON c.oid = i.inhrelid
JOIN pg_class p ON p.oid = i.inhparent
WHERE p.relname = 'events'
ORDER BY c.relname;

-- Monitor partition pruning effectiveness
SELECT
    query,
    calls,
    ROUND(mean_exec_time::numeric, 2) AS avg_ms
FROM pg_stat_statements
WHERE query ILIKE '%events%'
ORDER BY mean_exec_time DESC
LIMIT 10;
```

```sql
-- Check for queries NOT using partition pruning (seq scans on all partitions)
SELECT relname, seq_scan, seq_tup_read
FROM pg_stat_user_tables
WHERE relname LIKE 'events_%'
  AND seq_scan > 0
ORDER BY seq_scan DESC;

-- Partition count overview
SELECT
    p.relname AS parent_table,
    COUNT(c.oid) AS partition_count,
    pg_size_pretty(SUM(pg_total_relation_size(c.oid))) AS total_size
FROM pg_class p
JOIN pg_inherits i ON p.oid = i.inhparent
JOIN pg_class c ON c.oid = i.inhrelid
WHERE p.relkind = 'p'
GROUP BY p.relname
ORDER BY SUM(pg_total_relation_size(c.oid)) DESC;
```

Common Mistakes

1. **Including the partition key in a function call** — `WHERE DATE_TRUNC('month', created_at) = '2024-02-01'` prevents partition pruning. Compare the column directly.
2. **Unique constraints without partition key** — PostgreSQL cannot enforce uniqueness across partitions unless the unique constraint includes the partition key. Design around this limitation.
3. **Forgetting the DEFAULT partition** — Without it, inserts with unmatched values fail with an error. Always add a default partition.
4. **Not pre-creating future partitions** — Inserts to a range-partitioned table fail if no partition covers the new value. Use pg_partman or a scheduled job to create partitions in advance.
5. **Partitioning small tables** — Partitioning adds query planning overhead. Tables under a few million rows rarely benefit; the overhead (partition pruning logic) can make queries slower.
6. **Cross-partition updates that change the partition key** — `UPDATE events SET created_at = '2024-03-15' WHERE id = 123` moves a row from one partition to another (DELETE + INSERT). This is expensive and can cause lock contention.
7. **Missing indexes on child partitions** — Indexes defined on the parent propagate, but indexes created directly on individual child partitions don't appear on the parent. Use parent-level `CREATE INDEX` statements.

Best Practices

1. **Always pre-create future partitions** — Use pg_partman or a cron job to create 3-4 months ahead.
2. **Use monthly granularity** — Weekly is too many partitions; yearly is too few for most time-series.
3. **Match indexes to query patterns** — Don't index every column; index what queries filter on after the partition key.
4. **Use `DETACH CONCURRENTLY`** (PostgreSQL 14+) — Avoids table-level lock during partition removal.

5. **Monitor default partition** — A growing default partition signals missing partitions.
6. **Use `PARTITION BY RANGE (created_at::date)`** — Simpler than TIMESTAMPTZ for date-based partitioning if time zone handling isn't needed.

Performance Considerations

```
-- Benchmark: partitioned vs non-partitioned delete
-- Non-partitioned: DELETE FROM old_events WHERE created_at < '2024-01-01';
-- → Full table scan, generates WAL, slow

-- Partitioned: DROP TABLE events_2023_12;
-- → Metadata operation, no WAL for data, instant

-- Check constraint exclusion (legacy, pre-PG10)
SET constraint_exclusion = partition;  -- legacy, not needed with declarative
partitioning

-- Check planner overhead for many partitions
SHOW from_collapse_limit;  -- default 8
-- If you have 100+ partitions and complex queries, this matters
-- Consider: SET from_collapse_limit = 16; for queries over many partitions

-- Parallel query across partitions
SET max_parallel_workers_per_gather = 4;
EXPLAIN SELECT COUNT(*) FROM events WHERE user_id = 42;
-- Should show Parallel Append across partitions
```

Interview Questions & Answers

Q1: What is the difference between declarative and inheritance-based partitioning?

A: Declarative partitioning (PostgreSQL 10+) uses `PARTITION BY` in `CREATE TABLE` and child tables with `PARTITION OF`. The planner natively understands partition boundaries and applies partition pruning automatically. Inheritance-based (older approach) uses `INHERITS` with `CHECK` constraints and requires `constraint_exclusion = partition` for pruning. Declarative is preferred: it supports more partition types, is faster, supports global indexes, and has better tooling.

Q2: Explain partition pruning and when it fails.

A: Partition pruning is the planner optimization that eliminates partitions that cannot contain rows matching the query's WHERE clause. It works when the WHERE clause directly compares the partition key to a literal, bound parameter, or stable expression. It fails when: (1) the partition key is wrapped in a function (`DATE_TRUNC(...)` on a TIMESTAMPTZ partition key), (2) the comparison is on a derived or computed expression, (3) `enable_partition_pruning = off`.

Q3: When would you choose hash partitioning over range or list?

A: Hash partitioning is chosen when: (1) there's no natural ordering (no time series, no categorical values), (2) you want uniform data distribution across N partitions to spread I/O, (3) you want to scale writes across

storage devices. Range/list are preferred when queries filter on the partition key (enabling pruning). With hash partitioning, every query scans all partitions unless filtering by the exact hash key value.

Q4: How do you add a column to a partitioned table?

A: `ALTER TABLE parent_table ADD COLUMN new_col type;` — this propagates to all child partitions automatically. You cannot add a column with a volatile default in PostgreSQL < 14 without a table rewrite; in PostgreSQL 14+ there's no table rewrite for volatile defaults. You can also add the column only to individual partitions before attaching them.

Q5: Can you have a primary key on a partitioned table that doesn't include the partition key?

A: No. In declarative partitioning, a primary key (or any unique constraint) must include all columns of the partition key. This is because PostgreSQL enforces uniqueness only within a single partition — it can't guarantee global uniqueness across partitions unless the unique key includes the partition key (so rows that would conflict are always in the same partition).

Q6: What is `DETACH CONCURRENTLY` and when is it important?

A: `ALTER TABLE parent DETACH PARTITION child CONCURRENTLY` (PostgreSQL 14+) detaches a partition without taking a `SHARE UPDATE EXCLUSIVE` lock on the parent table. The regular `DETACH PARTITION` takes a strong lock that blocks reads and writes. `DETACH CONCURRENTLY` is critical in high-traffic production systems where blocking the parent table (which could be serving millions of rows/sec) during partition rotation is unacceptable.

Q7: Explain partition-wise joins. When are they beneficial?

A: Partition-wise joins work when two tables are partitioned using the same partition key and have matching partition boundaries. Instead of joining all partitions from table A against all partitions from table B, the planner joins matching partition pairs (partition 1 of A with partition 1 of B, etc.). This reduces memory usage, enables parallelism per partition, and improves cache efficiency. Enable with `SET enable_partitionwise_join = on`. Beneficial for large-scale analytics joins on time-series data.

Q8: How does pg_partman help with partition management?

A: pg_partman automates the entire partition lifecycle: (1) pre-creates future partitions based on `p_premake` setting, (2) drops or detaches old partitions based on `retention` setting, (3) handles partition naming conventions, (4) works with both time-based and ID-based partitioning, (5) integrates with pg_cron for scheduled maintenance. Without pg_partman, you need custom scripts and cron jobs to manage partition creation and cleanup.

Q9: What happens when you INSERT into a partitioned table when no partition covers the value?

A: If there is a `DEFAULT` partition, the row goes there. If there is no default partition, PostgreSQL raises an error: `ERROR: no partition of relation "table" found for row`. This is why you should always have a default partition and monitor it to ensure missing partitions are caught early.

Q10: How do you convert an existing large table to a partitioned table without downtime?

A: The zero-downtime approach: (1) Create the new partitioned table structure. (2) Set up a trigger on the old table to dual-write to the new table. (3) Backfill data in batches with pagination. (4) Monitor until new table is caught up. (5) Swap table names in a transaction. (6) Remove the trigger. This approach uses logical replication or `pg_repack`-like tooling; the key is avoiding a long-running lock that blocks application.

Exercises and Solutions

Exercise 1

Create a `logs` table partitioned monthly by `logged_at`. Create partitions for Jan–Mar 2024. Insert test data and verify with `EXPLAIN ANALYZE` that querying February logs only scans the February partition.

Solution:

```
CREATE TABLE logs (
    id        BIGSERIAL,
    severity  TEXT,
    message   TEXT,
    logged_at TIMESTAMPTZ NOT NULL DEFAULT now()
) PARTITION BY RANGE (logged_at);

CREATE TABLE logs_2024_01 PARTITION OF logs FOR VALUES FROM ('2024-01-01') TO
('2024-02-01');
CREATE TABLE logs_2024_02 PARTITION OF logs FOR VALUES FROM ('2024-02-01') TO
('2024-03-01');
CREATE TABLE logs_2024_03 PARTITION OF logs FOR VALUES FROM ('2024-03-01') TO
('2024-04-01');
CREATE TABLE logs_default PARTITION OF logs DEFAULT;

INSERT INTO logs (severity, message, logged_at)
SELECT 'INFO', 'Test message ' || g,
       '2024-01-01'::timestamptz + (g * interval '1 day')
FROM generate_series(1, 89) g;

EXPLAIN (ANALYZE, FORMAT TEXT)
SELECT COUNT(*) FROM logs WHERE logged_at >= '2024-02-01' AND logged_at < '2024-03-
01';
-- Should show: Seq Scan on logs_2024_02 (Partitions selected: 1 out of 3)
```

Exercise 2

Write a function that creates a partition for any given year and month, and also creates a GIN index on a `data` JSONB column of the parent table.

Solution:

```
CREATE OR REPLACE FUNCTION provision_partition(p_year INT, p_month INT)
RETURNS TEXT LANGUAGE plpgsql AS $$
DECLARE
    v_name TEXT := 'events_' || TO_CHAR(make_date(p_year, p_month, 1), 'YYYY_MM');
    v_start DATE := make_date(p_year, p_month, 1);
    v_end   DATE := v_start + INTERVAL '1 month';
BEGIN
    EXECUTE format(
        'CREATE TABLE IF NOT EXISTS %I PARTITION OF events
         FOR VALUES FROM (%L::timestamptz) TO (%L::timestamptz)',
```

```
        v_name, v_start, v_end
    );
    EXECUTE format('CREATE INDEX IF NOT EXISTS %I ON %I USING GIN (payload)',
        v_name || '_payload_gin', v_name);
    RETURN v_name;
END;
$$;

SELECT provision_partition(2024, m) FROM generate_series(5, 12) m;
```

Cross-References

- **08_query_optimization** — EXPLAIN ANALYZE for partition pruning verification
- **10_postgresql_internals** — TOAST, heap structure per partition
- **04_materialized_views.md** — Materialized views over partitioned tables
- **06_maintenance_procedures.md** (12_Production) — VACUUM strategy for partitioned tables
- **09_transactions_concurrency** — Lock implications of cross-partition updates
- **pg_partman docs** — https://github.com/pgpartman/pg_partman